INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

SISTEMAS DISTRIBUÍDOS

Python Threading e GIL

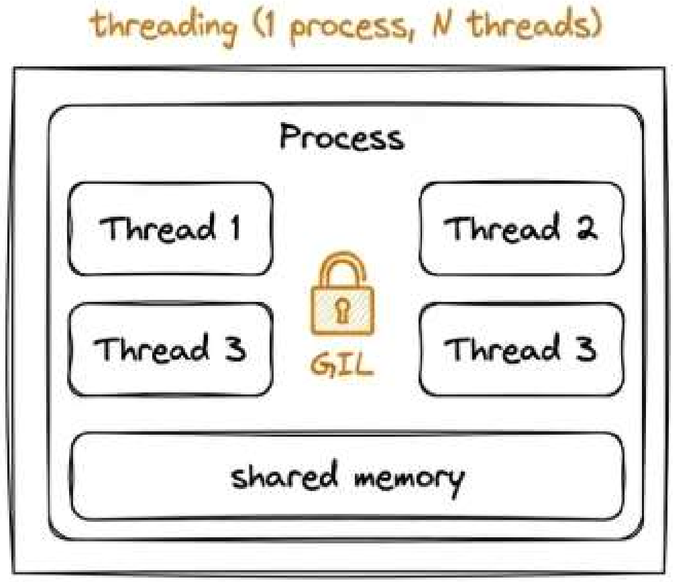


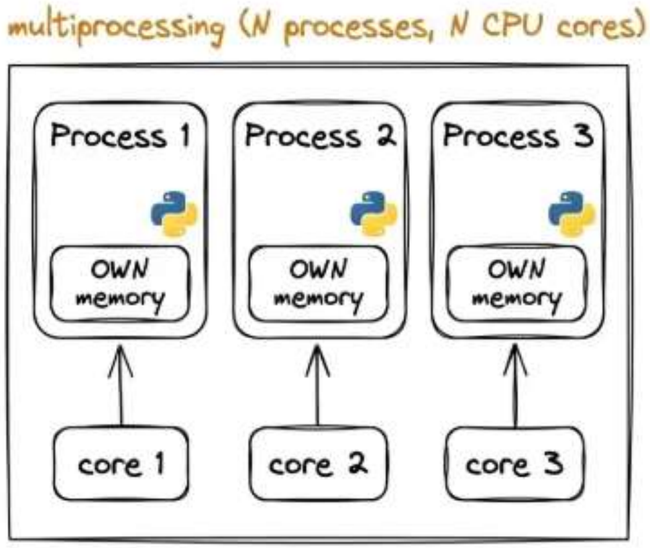
GABRIEL NUNES DE ALMDEIDA

Threads vs. processos

- A diferença fundamental entre ambos reside no isolamento e no gerenciamento de memória: o processo é uma unidade de execução independente enquanto a thread roda dentro de um processo, compartilhando a memória e os recursos com as demais threads.

	Thread	Processo
Memória	Compartilhada no mesmo processo	Espaço próprio e isolado
Criação	Leve e rápida	Mais pesada
Comunicação	Variáveis compartilhadas (exige sincronização)	IPC: pipes, filas, memória explícita
Isolamento	Baixo: falha grave pode derrubar tudo	Alto: falhas não se propagam
Paralelismo	Limitado pelo GIL	Paralelismo real em vários núcleos



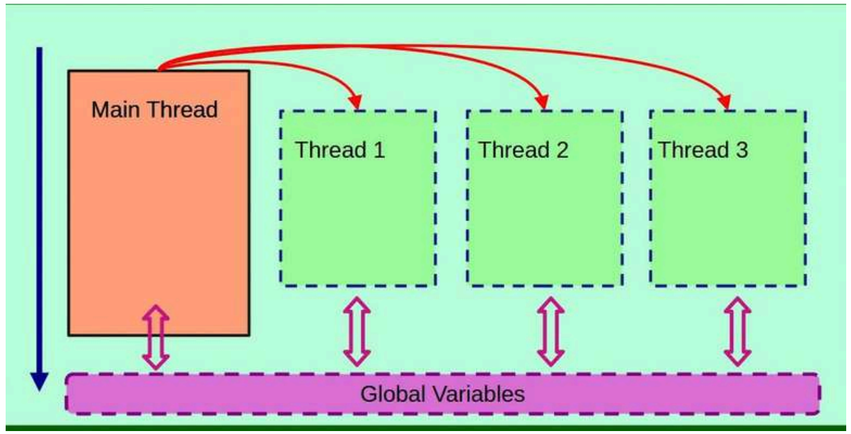


Asyncio vs. Threading

- **Ambos resolvem problemas de concorrência, mas com abordagens diferentes:**

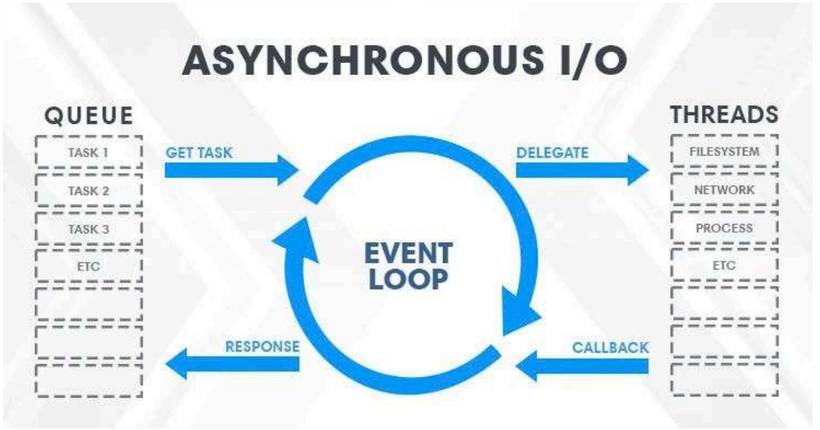
Threading

- É preemptivo. O Sistema Operacional decide quando pausar uma thread e iniciar outra (troca de contexto). Ideal para paralelizar operações de I/O síncronas/bloqueantes em código legado, sem precisar reescrever as funções.



Asyncio (Programação Assíncrona):

- É cooperativo e roda em uma única thread chamada Event Loop. O próprio código dita quando deve ceder o controle usando a palavra-chave await. É extremamente eficiente e consome menos memória, sendo a escolha moderna para aplicações de rede (APIs, WebSockets) com milhares de conexões simultâneas.

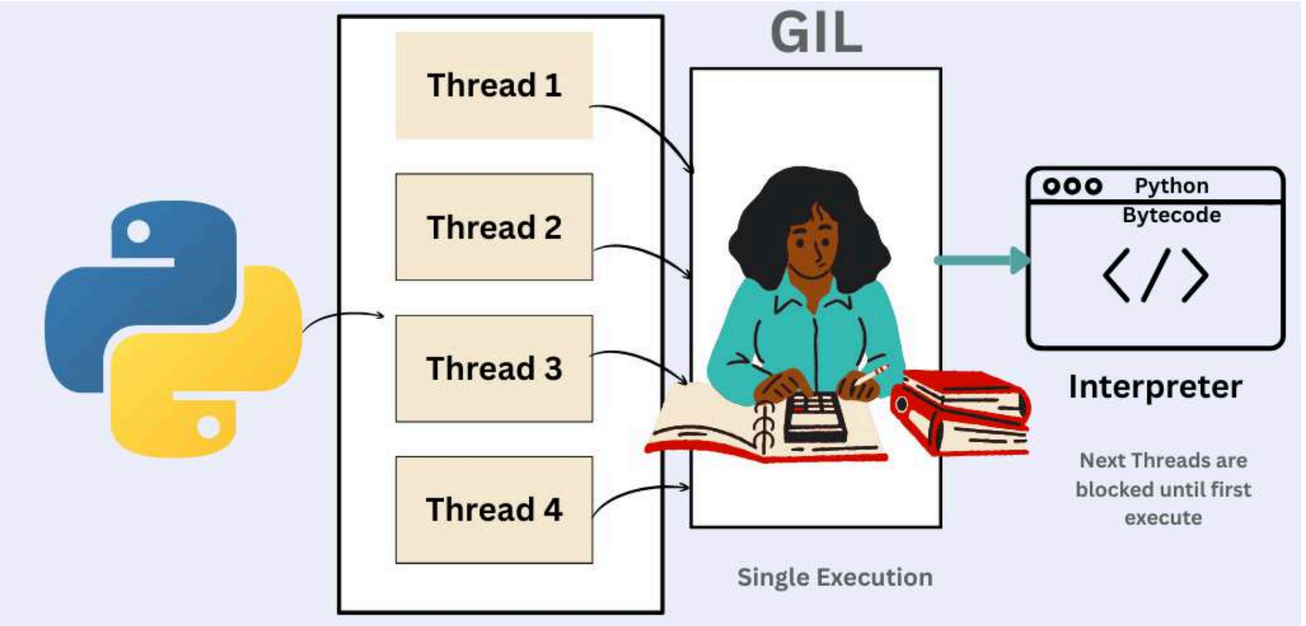


Python GIL (Global Interpreter Lock)

- **Definição: o GIL é um mutex global do CPython que permite que apenas uma thread execute bytecode Python por vez, mesmo em máquinas com vários núcleos.**

Por que existe:

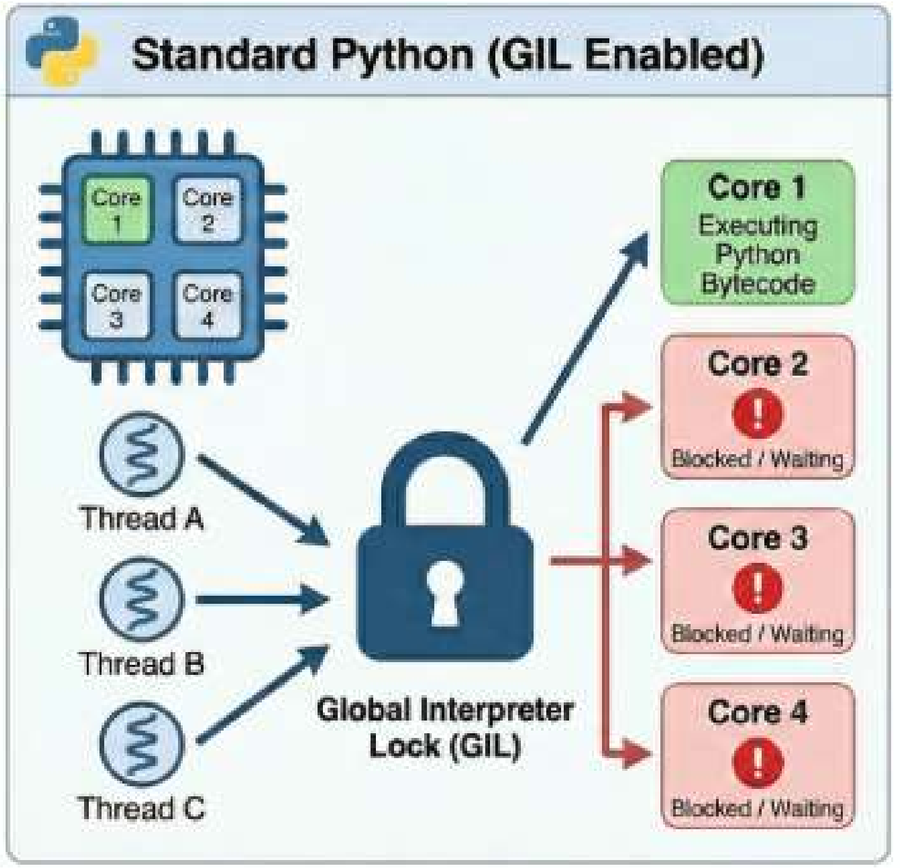
- **Gestão de memória**: Protege a contagem de referências contra corrupção e fugas de memória.
- **Desempenho single-thread**: Um lock único é mais simples e leve do que locks por objeto.
- **Extensões em C**: Simplifica a integração de módulos nativos ao evitar a concorrência no interpretador.
- **Contexto histórico**: Concebido na era dos processadores de um único núcleo, onde a solução era ideal.



Impacto do GIL no multithreading

- **Devido ao GIL, apenas uma thread pode executar código Python por vez em um processo CPython, não importando quantos núcleos o processador tenha.**

- **Tarefas CPU-bound:** threads não aceleram. Podem até ficar mais lentas, pelo custo da disputa pelo GIL.
- **Tarefas I/O-bound**: threads ajudam, porque o GIL é liberado enquanto se espera pelo I/O.
- **Extensões C que liberam o GIL** (NumPy em muitas operações, hashlib, zlib, I/O de arquivos e sockets) podem rodar em paralelo real.
- **O GIL não elimina race conditions**. Ele protege a integridade do interpretador, não a lógica do seu programa.



Quando threading é viável?

- **O multithreading é altamente viável e recomendado para tarefas I/O-bound (limitadas por entrada e saída). O interpretador Python é inteligente o suficiente para liberar o GIL enquanto uma thread realiza operações de bloqueio externo.**

Viável

- Requisições HTTP/APIs em paralelo
- Leitura/escrita de arquivos e acesso a banco de dados
- Web scraping
- Manter a interface gráfica responsiva enquanto algo roda em segundo plano
- Chamar bibliotecas bloqueantes que não têm versão async
- Código que usa extensões nativas que liberam o GIL

Inviável

- Cálculo numérico puro em Python
- Processamento de imagens em Python puro.
- Criptografia em Python puro.

Criação e execução de threads

```python
import threading
import time

def tarefa(nome, segundos):
    print(f"[{nome}] iniciando")
    time.sleep(segundos)  # O GIL é liberado durante a espera do time.sleep()
    print(f"[{nome}] finalizada")

# Instanciação das Threads:
t1 = threading.Thread(target=tarefa, args=("A", 2))
t2 = threading.Thread(target=tarefa, args=("B", 1), name="Worker-B")

# Inicia a execução assíncrona/concorrente das threads em segundo plano
t1.start()
t2.start()

# O método join() faz a thread aguardar até que t1 e t2 concluam a execução
t1.join()
t2.join()

print("Todas finalizadas")
```

Principais classes e métodos da biblioteca threading

Função	Uso
Thread(target, args, kwargs, name, daemon)	Construtor
Thread.start()	Inicia a execução da thread.
Thread.join()	Bloqueia até a thread terminar
Lock	Exclusão mútua
Event	Sinalização: set(), clear(), wait(), is_set()
Condition	Espera por uma condição: wait(), notify(), notify_all()
Timer	Executa uma função após um atraso
queue.Queue:	fila thread-safe, ideal para o padrão produtor/consumidor

Python 3.14 Free-Threading e Consequências

- Ambos resolvem problemas de concorrência, mas com abordagens diferentes:

O que mudou

- **Suporte Oficial (PEP 779)**: Deixa de ser um recurso experimental (Python 3.13) e passa a ser oficialmente suportado.
- **Build Dedicado (python3.14t)**: O GIL continua ativo por padrão; o modo free-threading é acessado via binário python3.14t.
- **Verificação em Tempo de Execução**: Nova função sys._is_gil_enabled() para checar se o GIL está ativo ou desativado.
- **Fallback de Segurança**: O GIL é reativado automaticamente caso sejam importadas extensões em C antigas ou incompatíveis.

Consequências

- **Paralelismo real de CPU**: Permite execução paralela em múltiplos núcleos usando apenas a biblioteca threading.
- **Impacto em single-thread e RAM**: Leve perda de desempenho (5% a 10%) em código de thread única e maior consumo de memória.
- **Exposição de race conditions**: Falhas de concorrência antes ocultadas pelo GIL passam a aparecer no código.

INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

SISTEMAS DISTRIBUÍDOS

Obrigado Pela Atenção



GABRIEL NUNES DE ALMDEIDA